ЗВІТ

Лабораторна робота №2

Профілювання продуктивності, аналіз оптимізації C++ та оцінювання енергоспоживання

Виконала: студентка групи ТТП-32 Тупчій Олександра

Мета: дослідити виконання обчислювальної програми засобами Linux, побудувати FlameGraph, порівняти компіляцію без оптимізації та з оптимізацією і проаналізувати споживання енергії.

Середовище: Acer Swift SF114-34; Intel Pentium Silver N6000; Windows та Ubuntu через WSL2; GCC; perf; GNU time; Brendan Gregg FlameGraph; Compiler Explorer; Windows PowerShell.

У звіті розділено вимірювання енергоспоживання всієї системи та наближену оцінку додаткових енерговитрат досліджуваної програми за даними батареї.

Опис досліджуваної програми

Програма створює масив із 1 000 000 чисел типу double. Функція processArray послідовно обчислює суму sqrt(data[i]) × 1.5. Виклик повторюється вказану кількість разів (у базових тестах — 300; в енергетичних — 18 000). Час обчислення вимірюється за допомогою std::chrono::steady_clock.

Вихідний файл: main.cpp. Команди компіляції:

```
g++ -O0 -g -fno-omit-frame-pointer main.cpp -o program_O0
g++ -O3 -march=native -g -fno-omit-frame-pointer main.cpp -o program_O3
```

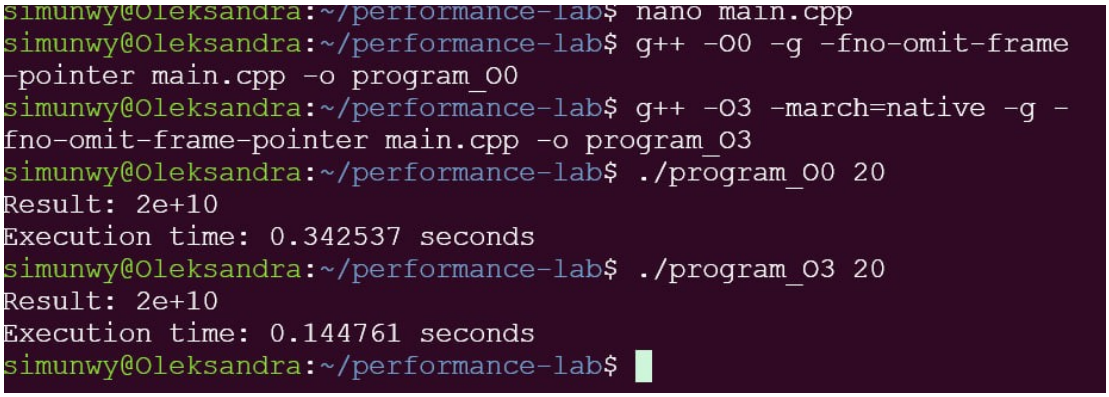


Рис. 1. Компіляція двох версій програми та короткий пробний запуск.

1. CPU FlameGraph: побудова та пояснення

FlameGraph — спосіб показати ієрархію викликів за результатами семплювального профілювання. Горизонтальна ширина прямокутників відповідає частці семплів, а вертикальне розташування демонструє стек викликів. Вісь X не є часовою шкалою.

Профілі отримано командою perf record із подією cpu-clock, частотою 99 Гц і збереженням стеку викликів. Далі perf script перетворено на згорнуті стеки за допомогою stackcollapse-perf.pl та візуалізовано flamegraph.pl.

```
perf record -e cpu-clock -F 99 -g --call-graph fp -o perf_O0.data -- ./program_O0 300
perf record -e cpu-clock -F 99 -g --call-graph fp -o perf_O3.data -- ./program_O3 300
```

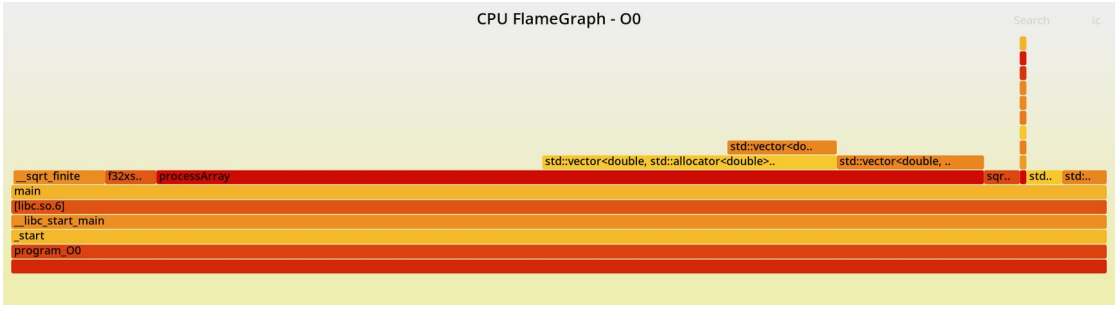


Рис. 2. FlameGraph без оптимізації (-O0).

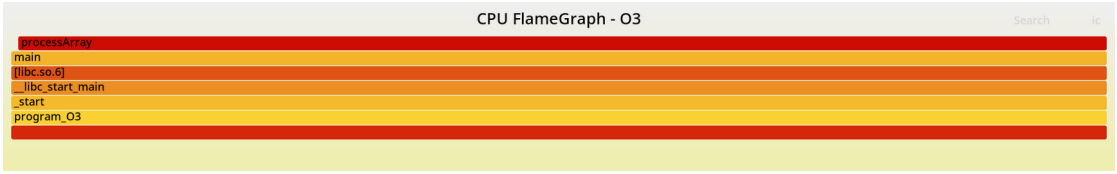


Рис. 3. FlameGraph після оптимізації (-O3).

У версії -O0 помітні окремі виклики processArray, std::vector::size та operator[]. У версії -O3 функція processArray вбудована в main; значна частина допоміжних викликів зникає. Скорочення кількості рівнів не означає автоматично скорочення абсолютного часу: для цього використовуємо незалежні вимірювання.

2. Аналіз продуктивності стандартними інструментами

2.1. /usr/bin/time --verbose

Утиліта надає реальний час, час CPU у режимах user/system, максимальний обсяг резидентної пам’яті (RSS), сторінкові помилки та кількість перемикань контексту.

Таблиця 1. Результати /usr/bin/time --verbose (300 ітерацій)

Показник	**-O0**	**-O3**	**Зміна**
Elapsed (wall clock)	4,96 с	1,92 с	−61,3 %
User CPU time	4,92 с	1,89 с	−61,6 %
System CPU time	0,02 с	0,02 с	≈ 0
Maximum RSS	11 920 КБ	11 912 КБ	≈ 0
Major page faults	4	0	−4
Minor page faults	2 107	2 112	≈ 0

```
simunwy@Oleksandra:~/performance-lab$ /usr/bin/time --verbose ./
program_O0 300
Result: 3e+11
Execution time: 4.87621 seconds
        Command being timed: "./program_O0 300"
        User time (seconds): 4.92
        System time (seconds): 0.02
        Percent of CPU this job got: 99%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:04.96
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 11920
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 4
        Minor (reclaiming a frame) page faults: 2107
        Voluntary context switches: 5
        Involuntary context switches: 12
        Swaps: 0
        File system inputs: 3608
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
simunwy@Oleksandra:~/performance-lab$
```

Рис. 4. /usr/bin/time --verbose для -O0.

```
simunwy@Oleksandra:~/performance-lab$ /usr/bin/time --verbose ./
program_O3 300
Result: 3e+11
Execution time: 1.89367 seconds
        Command being timed: "./program_O3 300"
        User time (seconds): 1.89
        System time (seconds): 0.02
        Percent of CPU this job got: 99%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.92
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 11912
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 2112
        Voluntary context switches: 1
        Involuntary context switches: 6
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
simunwy@Oleksandra:~/performance-lab$
```

Рис. 5. /usr/bin/time --verbose для -O3.

Основна різниця припадає на час роботи CPU; споживання пам’яті практично незмінне. Дані time та внутрішнього таймера програми можуть дещо відрізнятися через накладні витрати поза вимірюваною ділянкою.

2.2. perf stat -d

perf stat підраховує виконані інструкції, процесорні цикли, розгалуження й інші доступні події. У виводі використано апаратні події з мультиплексуванням, тому деякі лічильники оцінені за частиною інтервалу вимірювання (близько 66 %).

Таблиця 2. Результати perf stat -d (мультиплексування, ≈66 % інтервалу)

Показник	-O0	-O3	Зміна
Cycles	9 782 129 472	3 799 680 858	−61,2 %
Instructions	25 892 708 043	2 710 322 712	−89,5 %
Instructions / cycle	2,6	0,7	−73 %
Branches	3 903 542 900	604 211 632	−84,5 %
Branch misses	22 733	3 708	−83,7 %
Time elapsed	5,029 с	2,076 с	−58,7 %

Після оптимізації кількість виконаних інструкцій зменшилася приблизно на 89,5 %, а кількість циклів — на 61,2 %. Попри зниження IPC з 2,6 до 0,7, програма виконується швидше. Основний кандидат на обмеження IPC у -O3 — відносно дорога операція скалярного квадратного кореня sqrtsd; другорядний чинник — залежність послідовних addsd від значення акумулятора. Якщо поділити 3,8 млрд виміряних циклів на 300 млн опрацьованих елементів, отримаємо приблизно 12,7 циклу на елемент. Це середнє значення для всього запуску, а не окрема латентність sqrtsd; причина низького IPC є обґрунтованою гіпотезою, а не доведеним мікроархітектурним результатом.

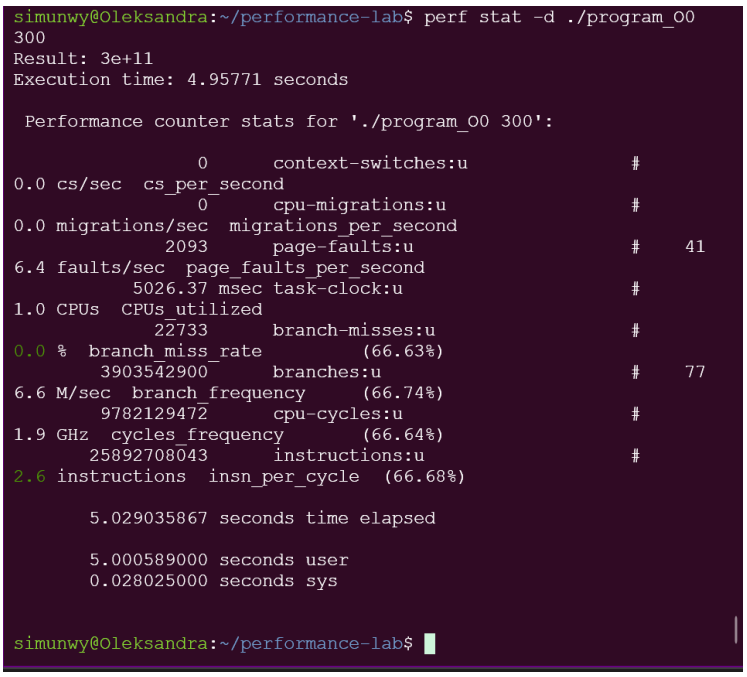


Рис. 6. perf stat -d для -O0.

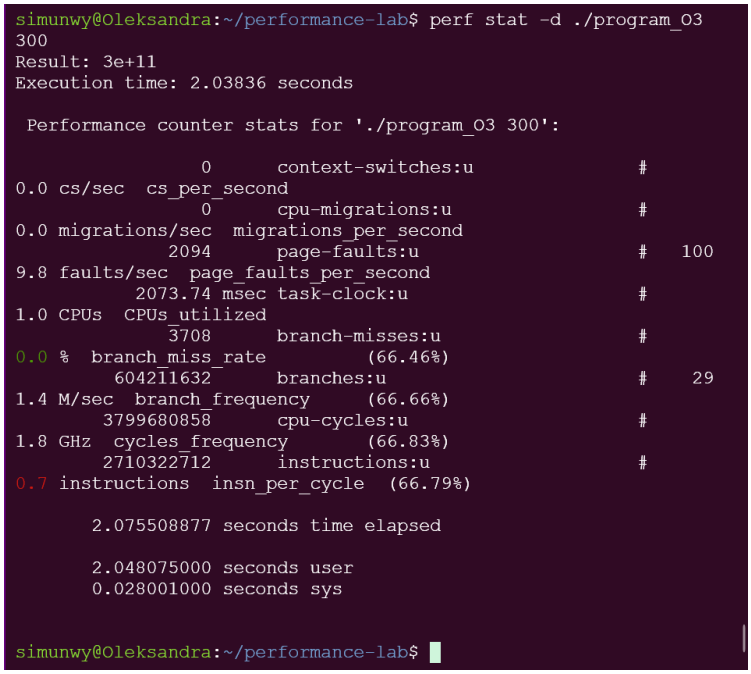


Рис. 7. perf stat -d для -O3.

2.3. perf record і perf report

perf record збирає семпли, а perf report дає змогу встановити, у яких функціях процесор проводить найбільшу частку часу. Зібрано 491 семпл для -O0 і 158 для -O3; втрачених семплів немає.

Для -O0 за часткою семплів переважають processArray (35,23 %), std::vector::size (27,49 %), std::vector::operator[] (20,16 %). Для -O3 профіль показує 100 % семплів у main, причому майже вся робота належить вбудованому processArray (99,37 % вкладених семплів). Це частки профілювання, а не точні частки фізичного часу окремих функцій.

Таблиця 3. Розподіл семплів perf report (частки профілювання)

Символ (за perf report)	**-O0**	**-O3**
processArray	35,23 %	вбудована в main (99,37 % вкладених семплів)
std::vector::size()	27,49 %	—
std::vector::operator[]	20,16 %	—
main (усі семпли)	—	100,00 %
Кількість семплів	491	158

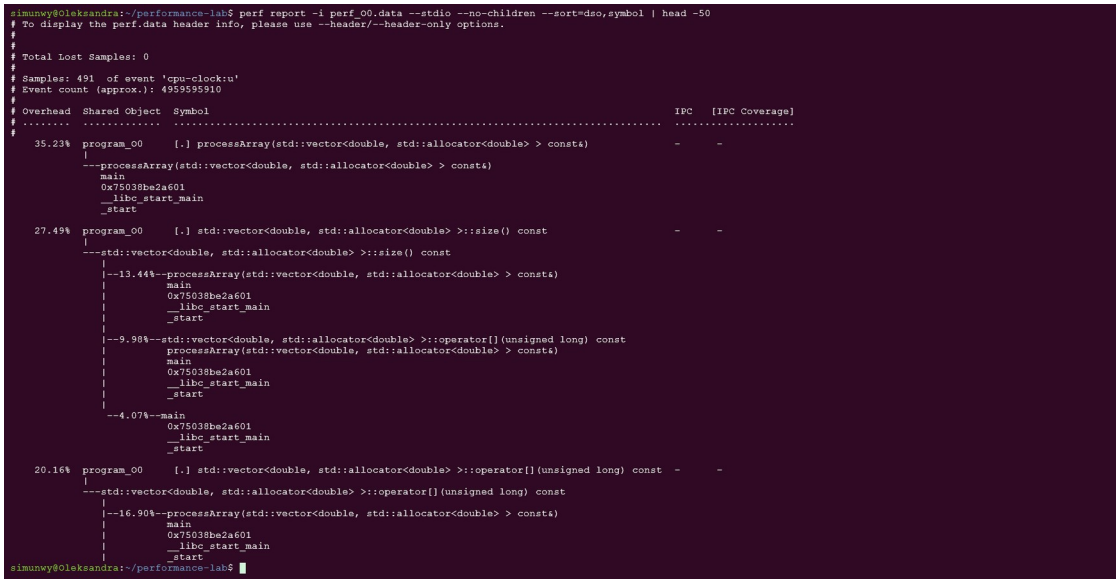


Рис. 8. Звіт perf report для -O0.

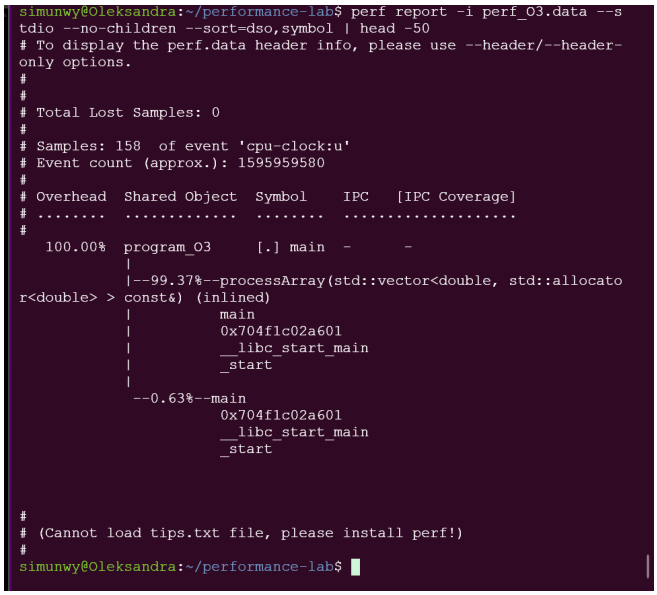


Рис. 9. Звіт perf report для -O3.

3. Оцінювання енергоспоживання

3.1. Енерговитрати всієї системи

У середовищі WSL2 недоступні потрібні лічильники RAPL. Тому використано Windows PowerShell та інтерфейс BatteryStatus: RemainingCapacity (мВт·год), DischargeRate (мВт). Перевірено сценарії простою, запуску -O0 та -O3 з однаковою кількістю повторень — 18 000. Енергія за зміною заряду батареї: E (Дж) = ΔCapacity (мВт·год) × 3,6. Середня потужність: P = E / t.

Таблиця 4. Енергоспоживання всієї системи (зміна заряду батареї)

Сценарій	Час, с	Енергія, Дж	Сер. потужність, Вт
Простій	121,975	1054,8	8,648
-O0 (18 000 повторень)	337,583	3520,8	10,429
-O3 (18 000 повторень)	97,036	1216,8	12,540
Зміна -O3 відносно -O0	**−71,3 %**	**−65,4 %**	**+20,2 %**

Загальна витрата енергії системи при -O3 зменшилася приблизно на 65,4 % порівняно з -O0. Хоча середня потужність у -O3 могла бути більшою, значно коротший час виконання зменшив загальну енергію за завершене завдання.

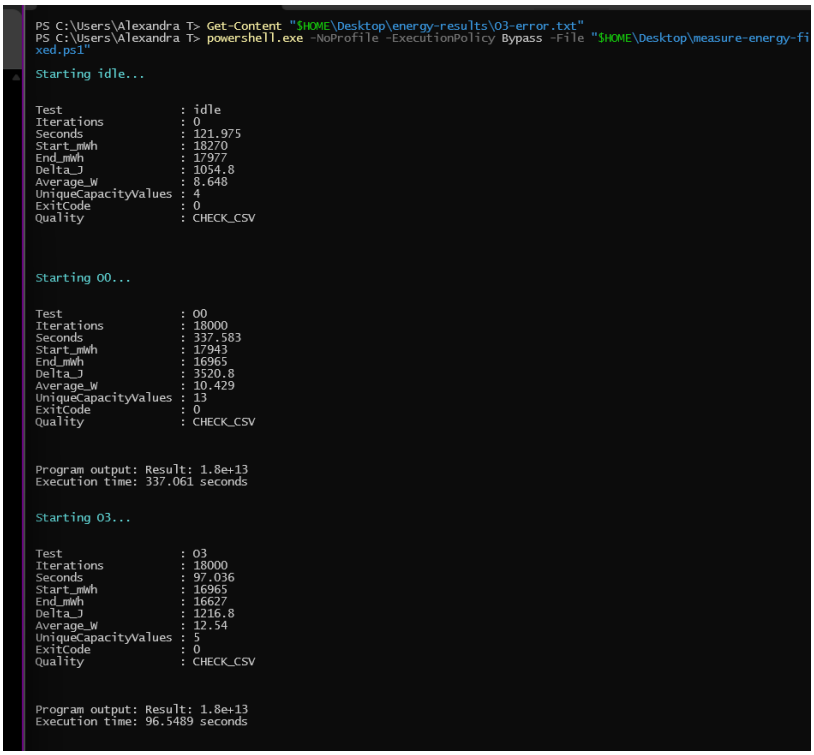


Рис. 10. PowerShell: простій, -O0 та -O3 в енергетичному експерименті.

3.2. Енерговитрати досліджуваного процесу та SRUM

Для оцінювання енерговитрат досліджуваної програми використано результати CSV-експерименту, отримані зі зміни заряду батареї Windows. Цей метод безпосередньо вимірює енергію всієї системи; внесок програми оцінено шляхом віднімання фонового споживання у стані простою.

За даними експерименту під час простою середня потужність становила 8,648 Вт (1054,8 Дж за 121,975 с). Для однакового обсягу обчислень у 18 000 повторень отримано: -O0 — 3520,8 Дж за 337,583 с; -O3 — 1216,8 Дж за 97,036 с.

Для наближеної оцінки додаткової енергії, пов’язаної з виконанням програми, використано формулу Епрогр ≈ Есист − Рпростою × t. За нею для -O0 отримано приблизно 601 Дж, а для -O3 — приблизно 378 Дж. Отже, оцінений додатковий енергетичний внесок під час виконання оптимізованої версії менший приблизно на 37 %. Ці числа є результатом розрахунку за експериментальними CSV-даними, а не прямого вимірювання окремого процесу.

Таблиця 5. Орієнтовна оцінка додаткової енергії програми (за даними з табл. 4)

Версія	E_сист, Дж	P_простою × t, Дж	E_прогр ≈ E_сист − P_простою × t, Дж
-O0	3520,8	2919,4	601
-O3	1216,8	839,2	378
Зміна -O3 відносно -O0			**−37 %**

3.2.1. Додаткова перевірка: Windows SRUM

Для незалежного додаткового спостереження використано Windows System Resource Usage Monitor (SRUM). Експорт виконано командою powercfg /srumutil у файл srum-tomorrow.csv. У журналі виявлено записи vmmemWSL, пов’язані із середовищем WSL2, у якому запускалися досліджувані програми.

Контрольні запуски виконано 21.10.2026. Відповідні інтервали в журналі SRUM (за інтерпретації його позначок часу як UTC) наведено в таблиці 6.

Таблиця 6. Контрольні запуски та інтервали в журналі SRUM

Версія	Час запуску (Київ)	Інтервал у журналі SRUM (якщо UTC)
-O3	22:36:43–22:38:30	19:36–19:38
-O0	22:48:02–22:53:59	19:48–19:54

У SRUM зафіксовано, зокрема, записи vmmemWSL, наведені в таблиці 7.

Таблиця 7. Записи vmmemWSL у експорті SRUM

Час запису	Тривалість інтервалу, с	TotalEnergyConsumption (одиниці поля SRUM)
21.10, 19:41	604,624	1 442
21.10, 19:41	1 134,430	316 948
21.10, 20:42:47	3 714,834	622 726

Ці значення наведено в одиницях вихідного поля SRUM без перерахунку в джоулі: одиниці та правила атрибуції у використаному експорті не підтверджені. Крім того, SRUM обліковує середовище vmmemWSL за довшими часовими інтервалами та може включати фонову активність, тому його дані не слід ототожнювати з енергією кожного окремого Linux-процесу.

Висновок до пункту 3.2. На основі CSV-експерименту отримано орієнтовні оцінки додаткового енергоспоживання програми: близько 601 Дж для -O0 та 378 Дж для -O3. SRUM додатково підтвердив наявність обліку енергетичної активності середовища WSL2. Таким чином, енергетичні дані отримано й проаналізовано, але наведені оцінки мають обмежену точність через змінне фонове споживання та особливості обліку WSL2.

4. Порівняння компіляторних оптимізацій

4.1. Асемблер -O0 та -O3

У -O0 функція processArray існує окремо; всередині циклу викликаються operator[], sqrt@plt та size(). Після -O3 тіло функції вбудовується у main, доступ до даних організовано через послідовний перехід по пам’яті, а для додатних елементів використовується апаратна інструкція sqrtsd. Операції множення й додавання залишаються скалярними (mulsd, addsd).

Фрагменти нижче — реальні рядки objdump із файлів assembly_O0.txt і assembly_O3.txt (адреси скорочено до змістовного циклу).

Лістинг 1. Фрагмент -O0: processArray

```
131a: call std::vector<double>::operator[]
1327: call sqrt@plt
1334: mulsd xmm0,xmm1
133d: addsd xmm0,xmm1
1352: call std::vector<double>::size()
1360: jne 130c <processArray+0x23>
```

Лістинг 2. Фрагмент -O3: цикл у main

```
12c0: movsd xmm0,QWORD PTR [rax]
12c4: ucomisd xmm3,xmm0
12c8: ja 13da <main+0x1fa>
12ce: sqrtsd xmm0,xmm0
12d9: mulsd xmm0,xmm2
12dd: addsd xmm1,xmm0
12e1: jne 12c0 <main+0xe0>
   ...
13da: mov QWORD PTR [rbp-0x58],rdx
13ef: call 1110 <sqrt@plt>
```

Перевірка ucomisd та умовний перехід ja ведуть до запасного шляху з викликом sqrt@plt для відповідних спеціальних значень. У звичайному циклі для додатних чисел виконується sqrtsd. Це узгоджується зі збереженням стандартної семантики sqrt без -ffast-math.

Таким чином, головна перевага -O3 полягає не в обов’язковому застосуванні AVX, а в інлайнингу, усуненні накладних викликів та спрощенні доступу до елементів.

4.1.1. SIMD та обмеження AVX

Прапорці процесора Intel Pentium Silver N6000, видимі в Ubuntu, містять SSE2/SSE4.x, але не містять AVX/AVX2. Тому локально виконувати AVX2-код у цьому експерименті не можна. Для окремої демонстрації використано Compiler Explorer та цикл додавання двох масивів float.

За -O0 згенеровано скалярні інструкції addss. За -O3 -mavx2 GCC створює vmovups ymm та vaddps ymm: 256-бітна інструкція vaddps обробляє вісім чисел float одночасно. Це демонстрація можливості векторизації, а не результат фактичного AVX-виконання на ноутбуці.

У самій досліджуваній програмі версія -O3 без -ffast-math не містить упакованих SIMD-операцій sqrtpd або addpd у головному циклі: спостерігаються скалярні sqrtsd, mulsd і addsd. Векторизацію можуть обмежувати вимоги до порядку додавання double під час редукції (без дозволу на переасоціацію), а також стандартна семантика sqrt, зокрема обробка помилок для недопустимих аргументів. Цю гіпотезу додатково перевірено на тій самій програмі: після компіляції з -O3 -march=native -ffast-math у реальному асемблері з'явилися sqrtpd, mulpd та addpd — двоелементні SSE2-операції над double у 128-бітних XMM-регістрах. Отже, SIMD-векторизацію в цьому окремому варіанті підтверджено, хоча AVX/AVX2 не використовуються. Параметр -ffast-math послаблює стандартні гарантії арифметики з плаваючою комою, тому програму program_fast не змішано з основним порівнянням -O0 і -O3.

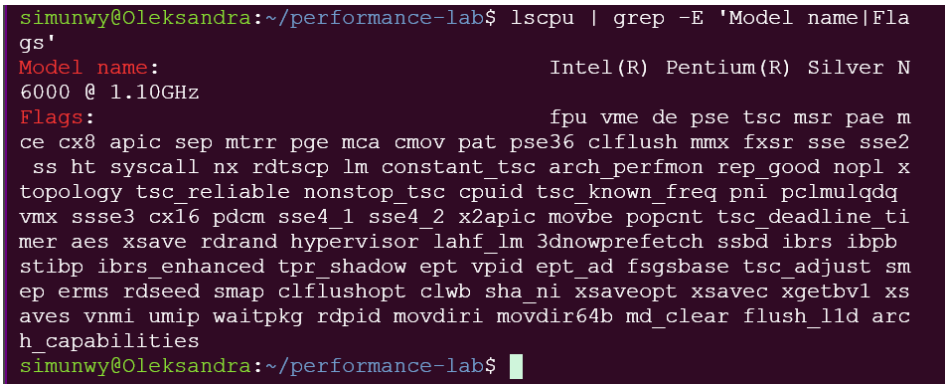


Рис. 11. lscpu: AVX/AVX2 у переліку прапорців відсутні.

4.1.2. Додатковий експеримент: SIMD без AVX на реальній програмі

```
g++ -O3 -march=native -ffast-math -g -fno-omit-frame-pointer main.cpp -o program_fast
objdump -d -C -Mintel program_fast | grep -E "sqrtpd|mulpd|addpd|sqrtsd"
```

У лістингу видно sqrtpd xmm0,xmm0; mulpd xmm0,xmm2; addpd xmm1,xmm0 — це справжні упаковані інструкції SIMD на досліджуваному Pentium Silver N6000. Присутність окремих sqrtsd також очікувана для залишкових елементів або допоміжних шляхів виконання.

Перший контрольний запуск ./program_fast 300 дав Result: 3e+11 та 1,88555 с за внутрішнім таймером. Для порівняння додатково проведено п'ять повторів за GNU time (таблиця у п. 4.2). SIMD є доведеним фактом на рівні асемблера, але виграш у швидкості не підтверджено. Векторні інструкції не гарантують прискорення: можливі обмеження пропускної здатності квадратного кореня, організації редукції та коливання частоти/навантаження. Конкретну причину цими вимірюваннями не встановлено.

Лістинг 3. Фрагмент фактичного SIMD-лістингу program_fast

```
15a0: movupd xmm3,XMMWORD PTR [rax]
15a4: add rax,0x10
15ab: sqrtpd xmm0,xmm3
15af: mulpd xmm0,xmm2
15b3: addpd xmm1,xmm0
15b7: jne 15a0 <processArray+0x40>
```

Фрагмент взято з assembly_fast.txt, функції processArray. Команда movupd завантажує дві сусідні double, а sqrtpd виконує пару квадратних коренів у XMM. Оскільки операції з плаваючою комою можуть бути перегруповані, результат такого варіанта не обов'язково є побітово тотожним базовому.

Таблиця 8. Ключові інструкції головного циклу

Операція	**-O0**	**-O3**	**-O3 -ffast-math**
Доступ до елемента	call operator[]	movsd (з пам’яті)	movupd (2 double)
Квадратний корінь	call sqrt@plt	sqrtsd (+ ucomisd/ja → sqrt@plt)	sqrtpd
Множення на 1.5	mulsd	mulsd	mulpd
Додавання до суми	addsd	addsd	addpd
Перевірка кінця циклу	call size()	jne за вказівником	jne за вказівником
Елементів за ітерацію	1	1	2

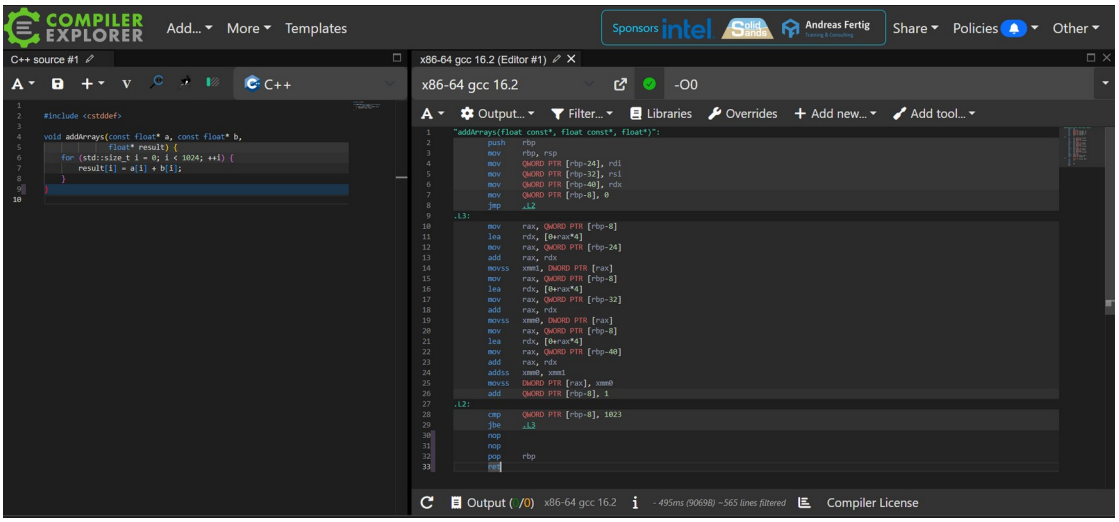


Рис. 12. Compiler Explorer: додавання масивів, -O0.

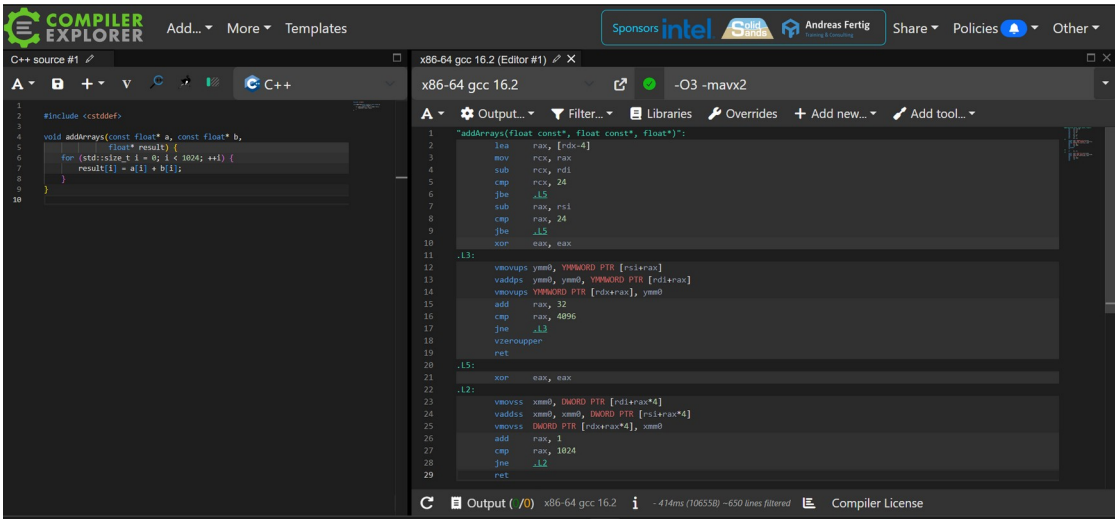


Рис. 13. Compiler Explorer: -O3 -mavx2, інструкція vaddps.

```
simunwy@Oleksandra:~/performance-lab$ cd ~/performance-lab

g++ -O3 -march=native -ffast-math -g -fno-omit-frame-pointer mai
n.cpp -o program_fast

objdump -d -C -Mintel program_fast | grep -E 'sqrtpd|mulpd|addpd
|sqrtsd' | head -30

./program_fast 300
    12ab:       66 0f 51 c0             sqrtpd xmm0,xmm0
    12af:       66 0f 59 c2             mulpd  xmm0,xmm2
    12b3:       66 0f 58 c8             addpd  xmm1,xmm0
    12bc:       66 0f 58 d9             addpd  xmm3,xmm1
    12d0:       66 0f 58 d3             addpd  xmm2,xmm3
    130b:       f2 0f 51 c0             sqrtsd xmm0,xmm0
    15ab:       66 0f 51 c3             sqrtpd xmm0,xmm3
    15af:       66 0f 59 c2             mulpd  xmm0,xmm2
    15b3:       66 0f 58 c8             addpd  xmm1,xmm0
    15c4:       66 0f 58 c1             addpd  xmm0,xmm1
    15d3:       f2 0f 51 c9             sqrtsd xmm1,xmm1
Result: 3e+11
Execution time: 1.88555 seconds
simunwy@Oleksandra:~/performance-lab$
```

Рис. 14. Локальний тест -ffast-math: SIMD-інструкції SSE2 та результат запуску.

```
simunwy@Oleksandra:~/performance-lab$ cd ~/performance-lab
mkdir -p results

for i in {1..5}; do
    /usr/bin/time -f "%e" -o "results/time_fast_$i.txt" ./progra
m_fast 300
done

cat results/time_fast_*.txt
Result: 3e+11
Execution time: 1.70187 seconds
Result: 3e+11
Execution time: 2.40536 seconds
Result: 3e+11
Execution time: 2.31901 seconds
Result: 3e+11
Execution time: 2.03897 seconds
Result: 3e+11
Execution time: 1.77951 seconds
1.74
2.43
2.33
2.06
1.79
```

Рис. 15. П’ять повторів запуску program_fast і показники часу.

4.2. Порівняння часу та кількісних метрик

Таблиця 9. Час виконання (300 повторень, 5 запусків за GNU time)

Повтор	-O0, с	-O3, с	-O3 -ffast-math, с
1	4,59	1,78	1,74
2	5,28	2,24	2,43
3	4,04	1,93	2,33
4	5,38	1,45	2,06
5	4,72	2,14	1,79
Середній час	**4,802**	**1,908**	**2,070**
Станд. відхилення, с	**0,547**	**0,313**	**0,310**
Прискорення відносно -O0	**—**	**2,52 ×**	**2,32 ×**

Прискорення за середніми значеннями: 4,802 / 1,908 ≈ 2,52 раза, або скорочення часу приблизно на 60,3 %. Вибіркове стандартне відхилення: 0,547 с (-O0) та 0,313 с (-O3). Кожний тест обробляв однакові вхідні дані та 300 повторень. Коливання між запусками можуть бути пов’язані з фоновою активністю, змінами частоти CPU та умовами WSL2.

Для -O3 -ffast-math середній час склав 2,070 с зі стандартним відхиленням 0,310 с. Це приблизно на 8,5 % більше за 1,908 с для -O3, але тести проведено різними серіями, на тлі помітного розкиду значень. Отже, фактичну SSE2-векторизацію підтверджено асемблером, а систематичного виграшу за часом не продемонстровано.

4.3. Повторне порівняння FlameGraph

На графіку -O0 окремо присутні стеки processArray, std::vector::size() та std::vector::operator[] — ці допоміжні виклики виконуються всередині циклу. Після компіляції з -O3 стовпчики size() та operator[] зникають як окремі функції: їхню логіку оптимізовано й інтегровано в цикл. processArray також вбудовується у main, тому графік -O3 стає компактнішим за ієрархією викликів. Це свідчить про зміну структури виконання, а прискорення незалежно підтверджують time та perf stat. Обидва графіки наведено в розділі 1; SVG-файли зберігаються окремо.

4.4. Порівняння енерговитрат до і після оптимізації

Загальне енергоспоживання системи під час -O3 було на 65,4 % меншим, ніж під час -O0 за однакового навантаження. Різниця пов’язана насамперед зі скороченням часу виконання. SRUM додатково показав облік vmmemWSL, проте не ізолював окремі запуски; тому точних процесних джоулів не наведено (див. 3.2).

Таблиця 10. Підсумкове порівняння -O0 та -O3

Показник	-O0	-O3	Зміна
Середній час (5 запусків)	4,802 с	1,908 с	−60,3 %
Інструкції (perf stat)	25,9 млрд	2,71 млрд	−89,5 %
Цикли (perf stat)	9,78 млрд	3,80 млрд	−61,2 %
Енергія системи (18 000 повторень)	3520,8 Дж	1216,8 Дж	−65,4 %
Оцінена додаткова енергія програми (орієнтовно, п. 3.2)	≈ 601 Дж	≈ 378 Дж	≈ −37 %

Висновки

Оптимізація GCC -O3 для досліджуваної програми скоротила середній час виконання приблизно у 2,52 раза. Профілювання й асемблер показали інлайнинг processArray, скорочення допоміжних викликів і кількості інструкцій. Показано побудову та інтерпретацію FlameGraph. AVX2 продемонстровано окремо на Compiler Explorer через апаратні обмеження Pentium Silver N6000; натомість реальну SSE2-векторизацію тієї самої програми підтверджено локально з -ffast-math. За п’ятьма додатковими запусками статистично переконливого прискорення від SIMD не встановлено. У вимірюваннях батареї сумарні витрати енергії системи зменшилися після оптимізації; точний внесок окремого процесу лишається невизначеним.

Файли для відтворення експерименту

Файл	**Призначення**
main.cpp	вихідний код програми
perf_O0.data, perf_O3.data	профілі perf record
flame_O0.svg, flame_O3.svg	FlameGraph для -O0 та -O3
assembly_O0.txt, assembly_O3.txt, assembly_fast.txt	асемблерний лістинг (objdump)
program_fast (локально)	збірка з -O3 -ffast-math
results/time_O0_*.txt, time_O3_*.txt, time_fast_*.txt	результати вимірювань часу
measure-energy-fixed.ps1	PowerShell-скрипт вимірювання енергії
idle.csv, O0.csv, O3.csv, summary.csv	дані вимірювання батареї
srum_wsl_public.csv	експорт Windows SRUM

Дані SRUM не прирівнюються до прямої енергії процесу.